

第1章 编译程序概述

伍春香

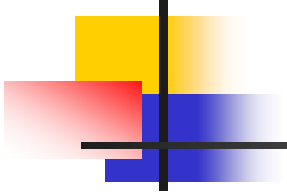
wuchxcourse@sina.com

武汉大学计算机学院

如某页有遮盖,
请查看前一页,
前一页无遮盖。

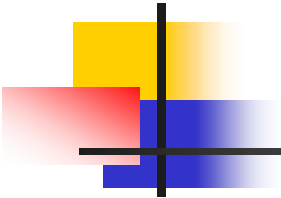
说 明

PPT为课堂讨论提纲，提前发到QQ群里供大家
课前预习、思考用，实际授课时会有增删改。



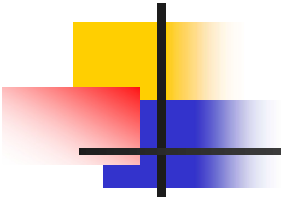
本章内容

- ◆ 程序的翻译
- ◆ 编译程序的工作内容
- ◆ 编译程序的结构
- ◆ 编译程序的组织方式
- ◆ 编译程序的构造



阅读内容

- 【ALSU】
 - 第1章
 - 1.1、 1.2、 1.3
- 【louden】
 - 第1章
 - 1.2、 1.3
- 【CLTZL】



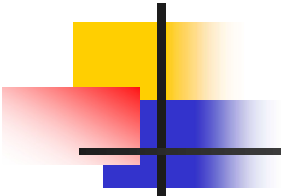
学习目标

- 重点掌握

- 能叙述程序翻译的基本思想，并能用其分析、解释相关问题
- 能叙述两种程序翻译方式及其特点，并能用其分析、解释相关问题
- 能叙述编译程序及其工作内容，并能用其分析、解释相关问题
- 能叙述编译程序的结构，并能用其分析、解释相关问题

- 一般了解

- 能叙述编译程序的构造方法



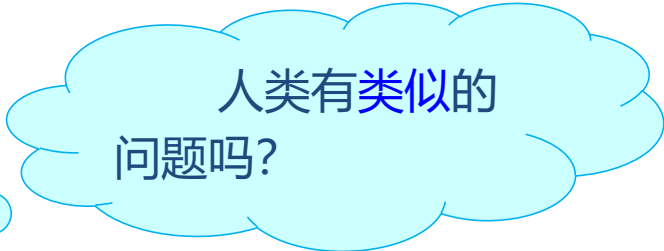
1.1 程序的翻译

- 程序设计语言

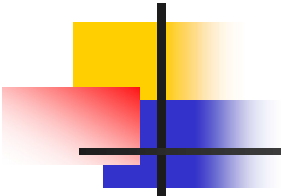
- 机器语言 C7 06 0000 0002
- 汇编语言 Mov x, 2
- 高级语言 x = 2

- 问题

- 计算机只能识别二进制数0、1表示的指令和数构成的本计算机系统的机器语言。如何让计算机执行高级语言程序呢？



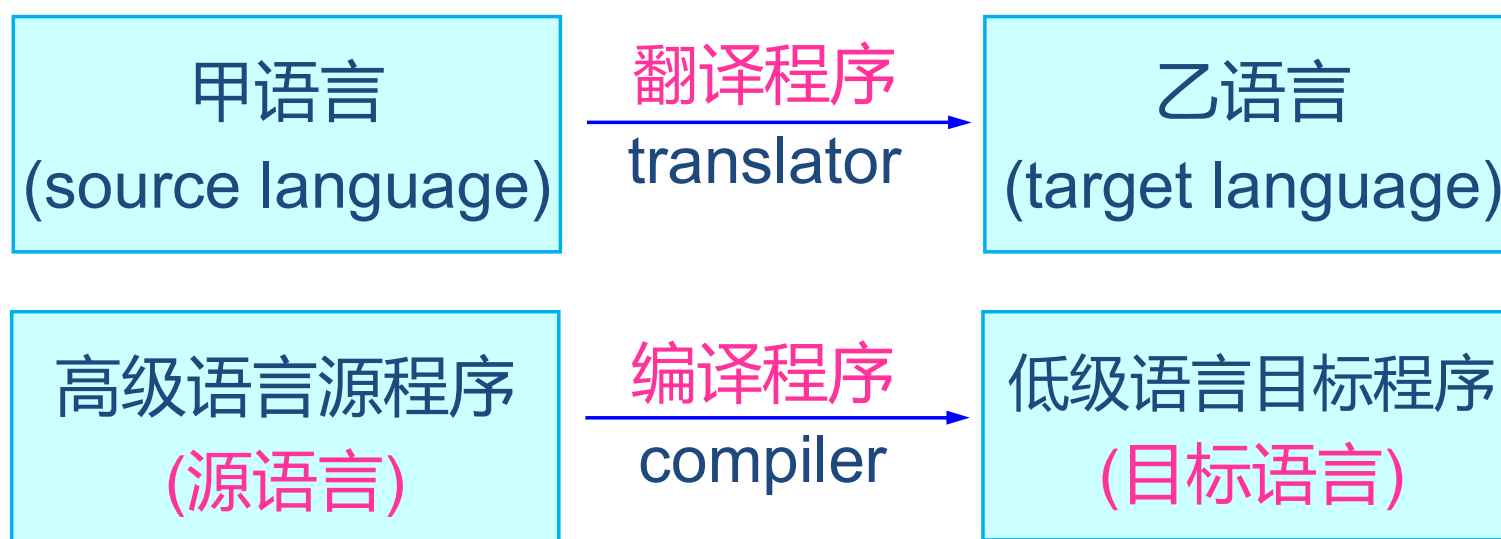
人类有类似的问题吗？

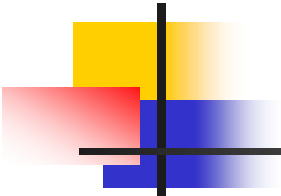


1.1 程序的翻译

- 翻译程序

- 所谓翻译程序是指这样一种程序，它能将用甲语言(源语言, source language)编写的程序翻译成与之等价的、用乙语言(目标语言, target language)书写的程序。





1.1 程序的翻译

- 翻译程序
 - 所谓翻译程序是指这样一种程序，它能将用甲语言(源语言, source language)编写的程序翻译成与之等价的、用乙语言(目标语言, target language)书写的程序。
- 程序的翻译通常有两种方式
 - “编译” 方式
 - “解释” 方式

1.1 程序的翻译

- 编译方式

- 编译方式是一种分阶段进行的方式

- 翻译阶段



- 运行阶段



1.1 程序的翻译

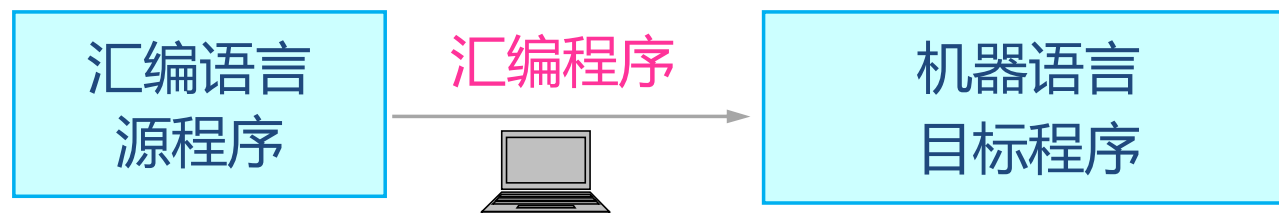
- 编译方式

- 编译方式下的翻译程序

- 编译程序

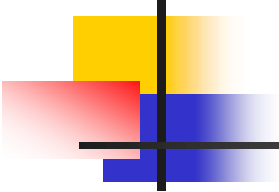


- 汇编程序



C7 06 0000 0002

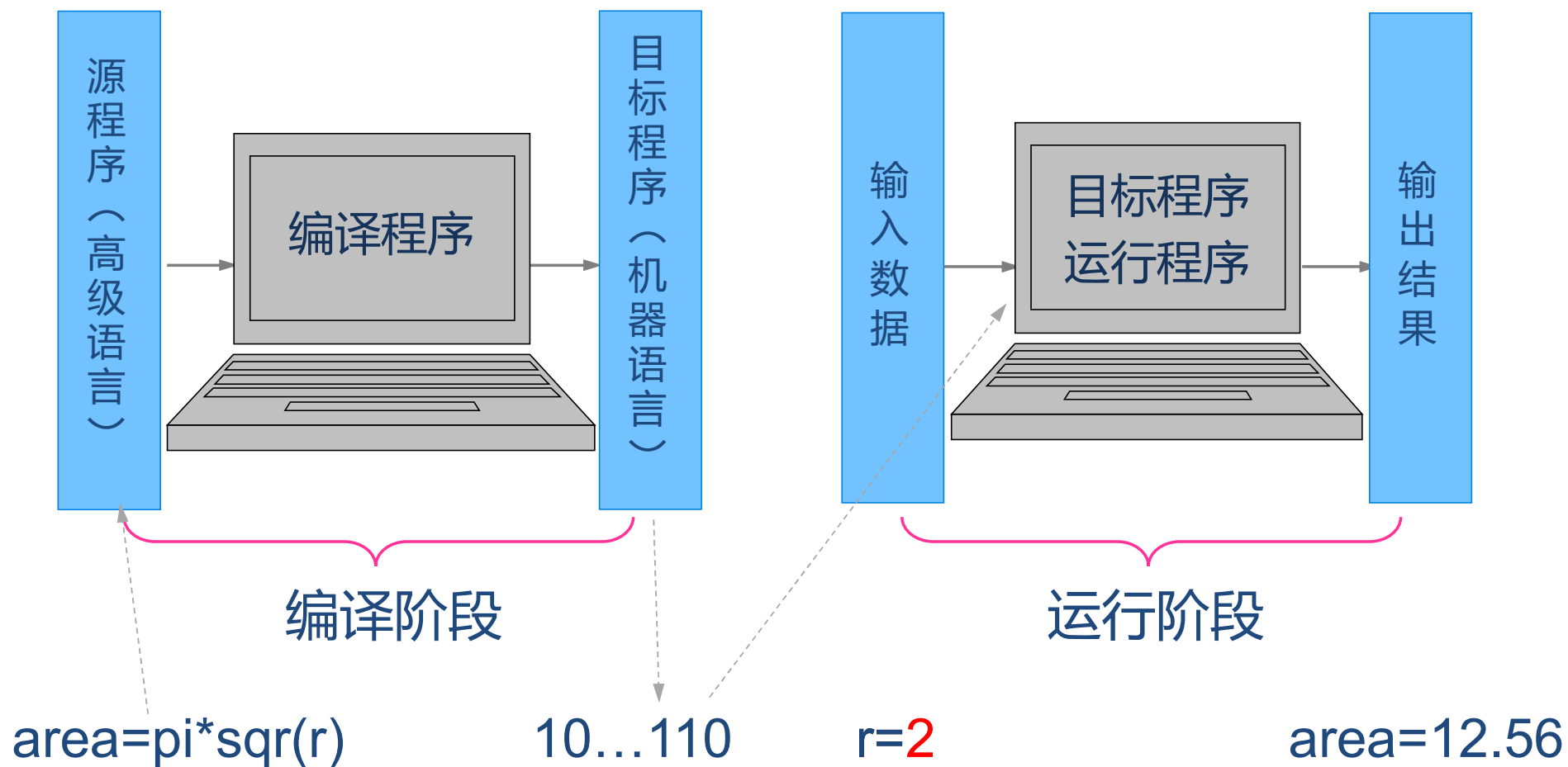
Mov x, 2

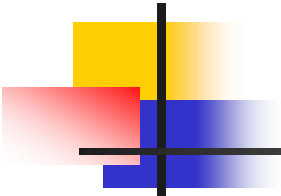


1.1 程序的翻译

- 编译方式的特点
 - 在编译方式下，源程序的执行需要分阶段
 - 如果目标程序是机器语言程序，则源程序的执行分为两大阶段：
编译阶段和运行阶段

分成两个阶段的源程序的执行历程

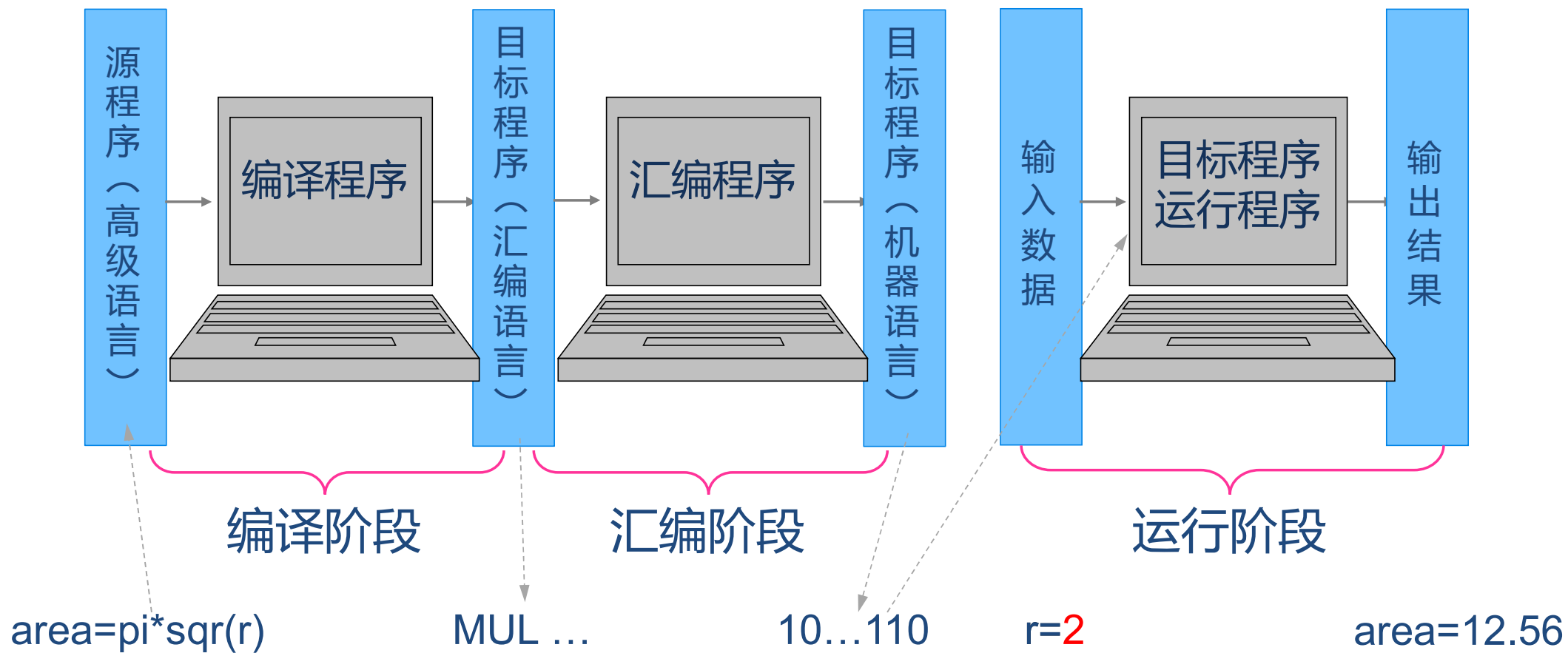


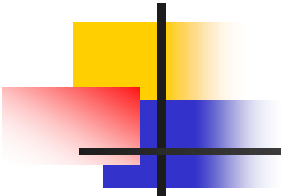


1.1 程序的翻译

- 编译方式的特点
 - 在编译方式下，源程序的执行需要分阶段
 - 如果目标程序是机器语言程序，则源程序的执行分为两大阶段：
编译阶段和运行阶段
 - 如果目标程序是汇编语言程序，则源程序的执行分为三大阶段：
编译阶段、汇编阶段和运行阶段

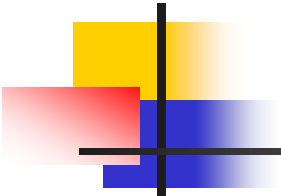
分成三个阶段的源程序的执行历程





1.1 程序的翻译

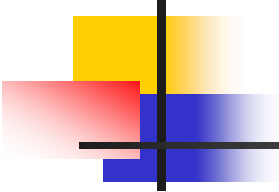
- 编译方式的特点
 - 在编译方式下，源程序的执行需要分阶段
 - 如果目标程序是机器语言程序，则源程序的执行分为两大阶段：
编译阶段和运行阶段
 - 如果目标程序是汇编语言程序，则源程序的执行分为三大阶段：
编译阶段、汇编阶段和运行阶段
 - 编译方式下，生成了目标代码，且可多次被使用



关于编译程序的几点说明

- 编译程序生成的目标程序不一定是机器语言的程序
 - 也有可能是汇编语言程序
- 编译程序与具体的机器和语言有关*
 - 即任何一个具体的编译程序都是某一特定类型的计算机系统中关于某一特定语言的编译程序
 - Eclipse
 - 通过不同的插件（JDT、CDT、PyDev等）分别集成或者调用 ECJ、GCC、Python 等不同的语言处理工具
- 对编译程序而言，源程序是输入数据，目标程序是输出结果

屏蔽了底层工具链的复杂性

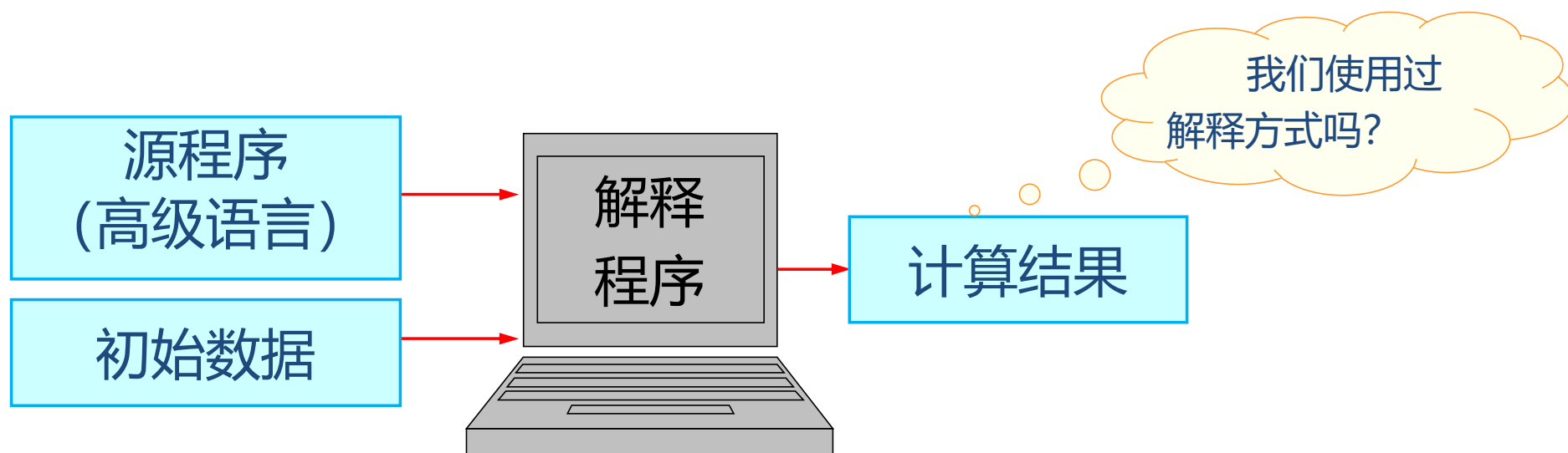


1.1 程序的翻译

- 解释方式

- 完成解释工作的解释程序将按源程序中语句的**动态顺序**，**逐句**地进行分析解释源程序，并立即予以执行

源程序解释执行的历程



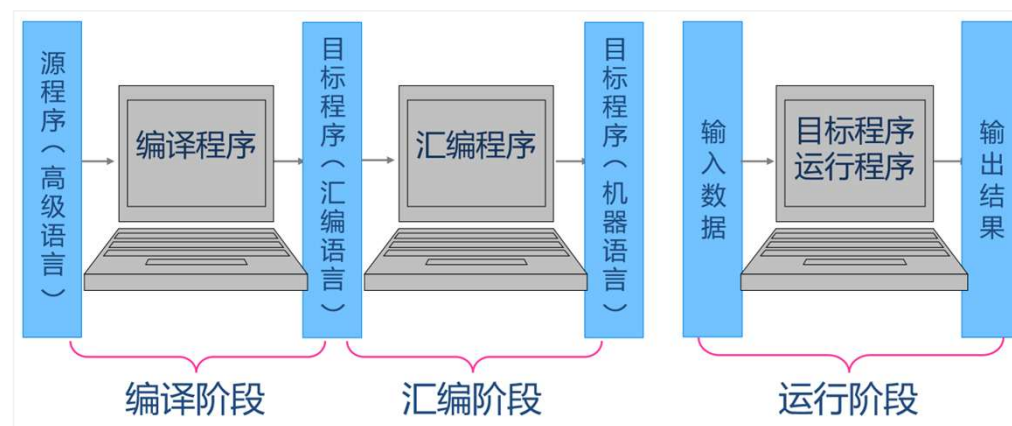
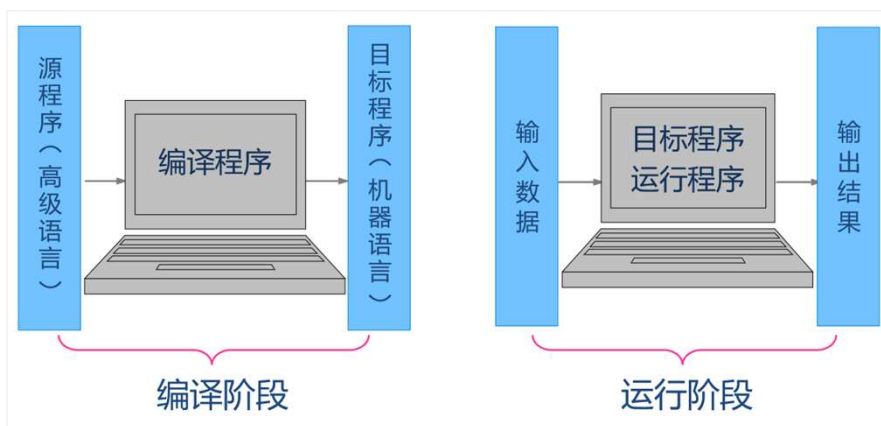
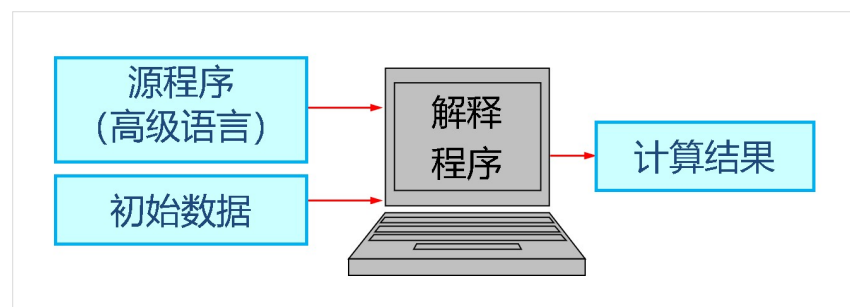
在解释方式下，并不生成目标代码，而是直接执行源程序本身。
这是编译方式与解释方式的根本区别。*

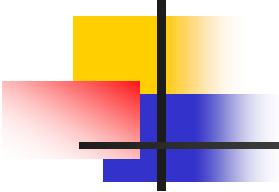
1.1 程序的翻译

两种翻译方式的
优缺点?

- 解释方式

- 完成解释工作的解释程序将按源程序中语句的**动态顺序**，**逐句**地进行分析解释源程序，并立即予以执行
- 解释方式与编译方式的根本区别
 - **是否生成了目标代码**

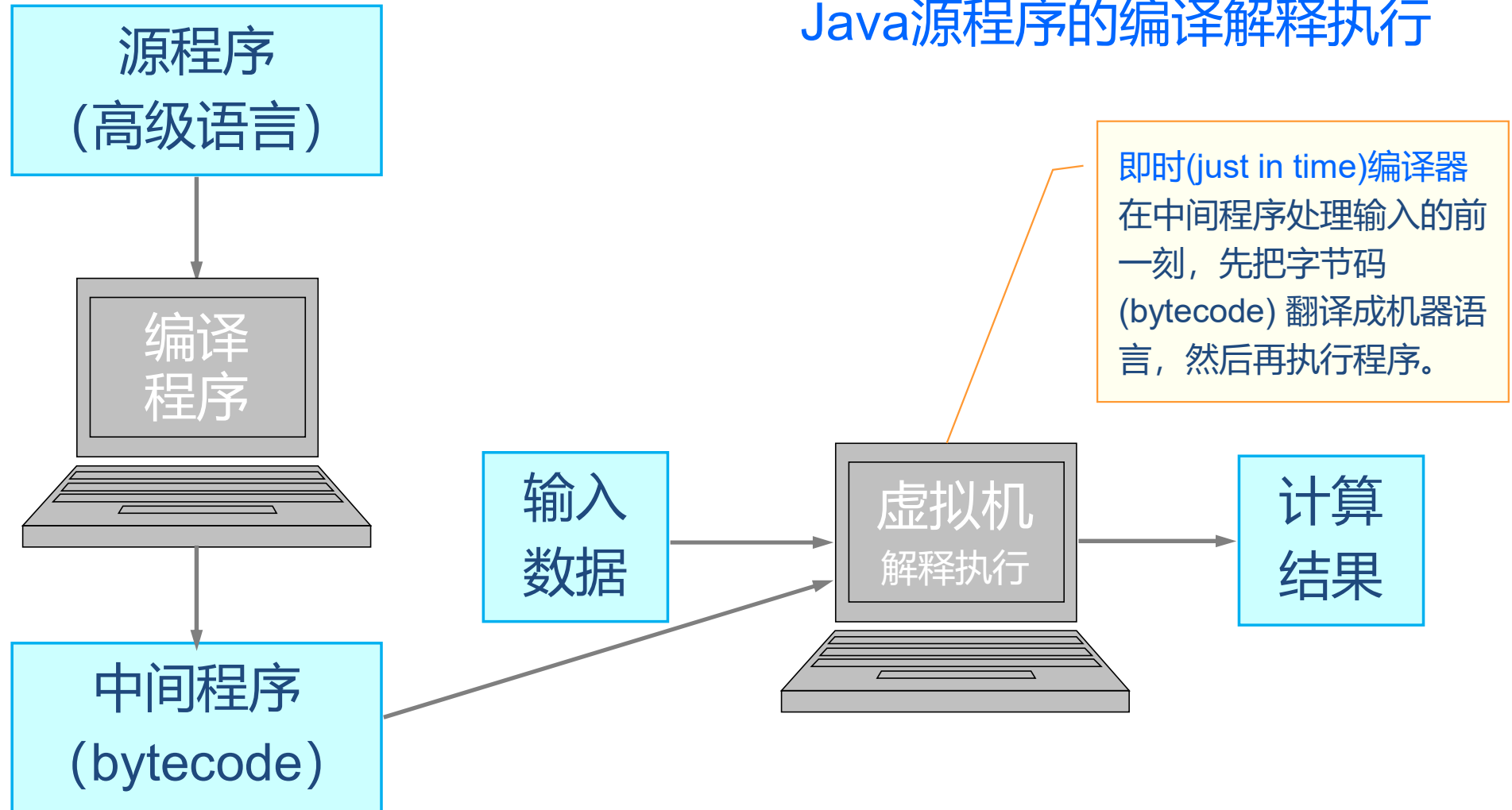


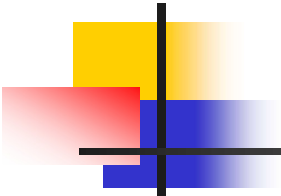


1.1 程序的翻译

- 混合方式
 - 兼有编译和解释过程
 - 如Java
 - ✓ 在一台机器上编译得到字节码(bytecode)
 - ✓ 由虚拟机对字节码加以解释执行
 - 字节码可以在另一台机器上解释执行

Java源程序的编译解释执行





编译器简史

- 第一个完整编译器

- 1954 ~ 1957
- IBM, John Backus团队
- FORTRAN

允许程序员以更自然和可读的方式编写代码——开发和
维护软件变得更加容易

机器语言 C7 06 0000 0002

汇编语言 Mov x, 2

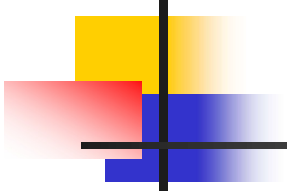
• 高级语言 x = 2

- 相关理论和算法

- 1960s ~ 1970s

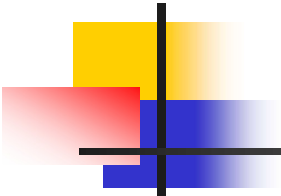
- 自动生成(部分功能)工具

- 1975年
 - Yacc, Lex



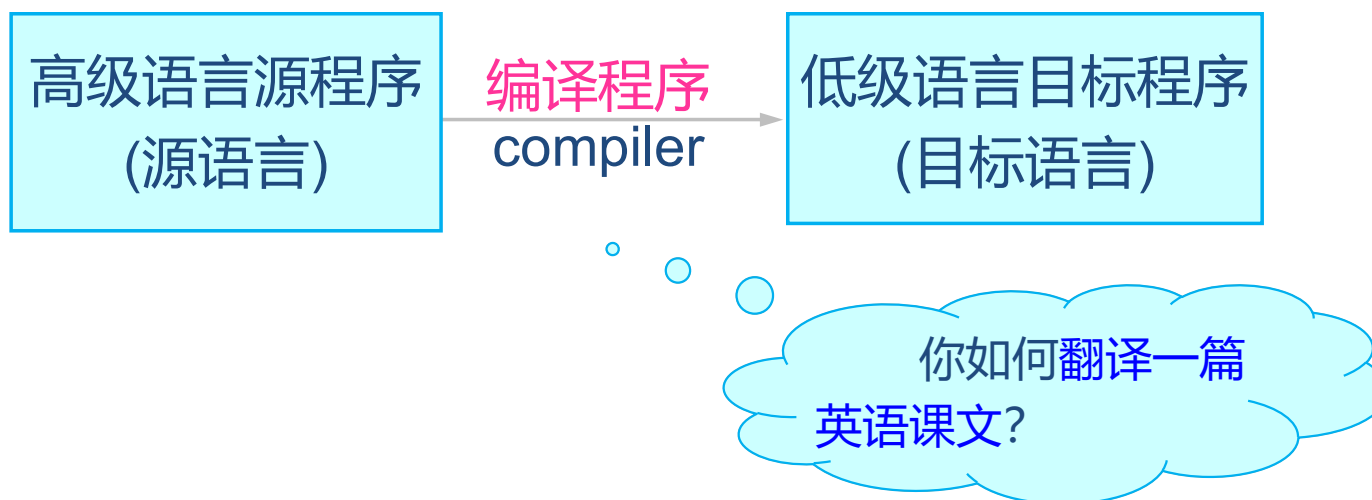
本章内容

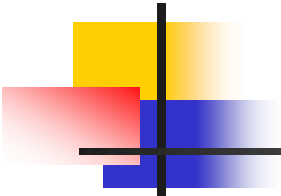
- ◆ 程序的翻译
- ◆ 编译程序的工作内容
- ◆ 编译程序的结构
- ◆ 编译程序的组织方式
- ◆ 编译程序的构造



1.2 编译程序的工作内容

- 编译程序的目的
 - 将高级语言的源程序翻译成等价的低级语言程序





1.2 编译程序的工作内容

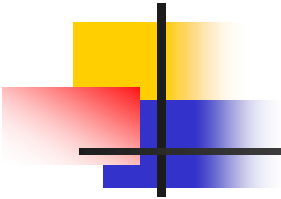
- 编译程序的目的

- 将高级语言的源程序翻译成等价的低级语言程序



- 编译程序的工作内容

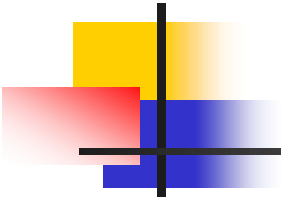
- 功能：词法分析、语法分析、语义分析、中间代码优化、目标代码生成



1.2 编译程序的工作内容

- 词法分析

- 依据语言词法规则，分析由字符组成的源程序，把它识别为一个一个具有独立意义的**最小语法单位**，即“单词 / 词素(lexeme)”，并识别出与其**相关的属性**（如是标识符，是界限符，还是数，等等），再转换成**长度上统一的标准形式**（这种统一的标准形式既刻画了单词本身，又刻画了它所具有的属性，称为属性字token），以供其它部分使用。



1.2 编译程序的工作内容

- 程序设计语言的词法规则举例

$\langle \text{标识符} \rangle ::= \langle \text{字母} \rangle \{ \langle \text{字母} \rangle | \langle \text{数字} \rangle \}$

$\langle \text{无正负号常数} \rangle ::= \langle \text{数字} \rangle \{ \langle \text{数字} \rangle \}$

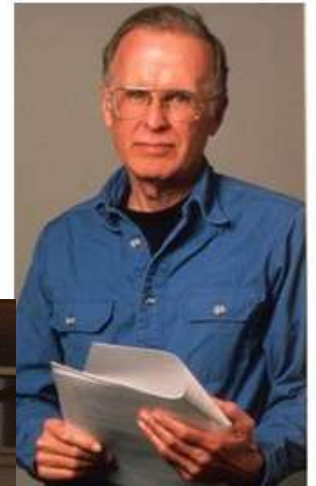
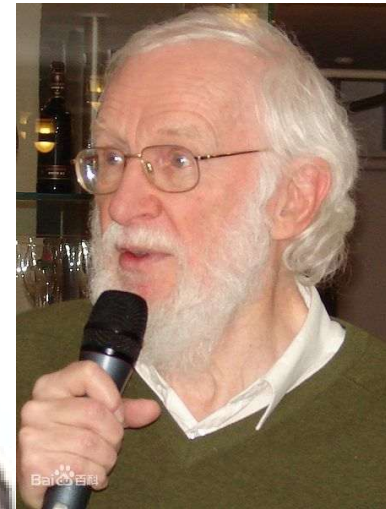
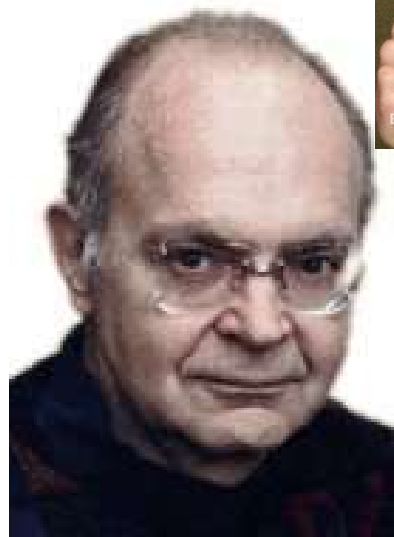
$\langle \text{字母} \rangle ::= a | b | c | \dots | x | y | z$

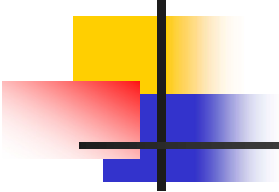
$\langle \text{数字} \rangle ::= 0 | 1 | 2 | \dots | 8 | 9$

EBNF (Extended
Backus-Naur Form)

Backus-Naur Form

- John Warner Backus
 - ALGOL 58
- Peter Naur
 - ALGOL 60
 - Backus Normal Form
- Donald Ervin Knuth
 - Backus–Naur Form





John Backus

BNF(Backus-Naur form) notation :

- ♦ 用尖括号 “<>”括住待描述的语法成分
- ♦ “::=”表示语法成分的定义
- ♦ “|”表示 “或者”

元符号

EBNF(extended Backus-Naur form):

- ♦ [x]—— x 可以出现零次或一次
- ♦ { x }—— x 可以出现零次到多次
- ♦ x(y | z)——等价于 xy 或 xz

G[<程序>]:

<程序>::=<程序首部> ; <分程序>.

<程序首部>::=program <标识符>

<分程序>::=<复合语句>

<复合语句>::=begin <赋值语句> end

<赋值语句>::=<标识符> := <表达式>

<表达式>::=<项> {(+|-) <项>}

<项>::=<因式> {(*/|) <因式>}

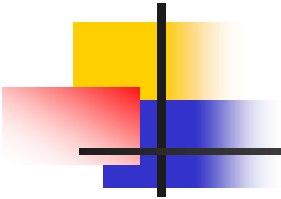
<因式>::=<标识符>|<无正负号常量>

<标识符>::=<字母>{<字母>|<数字>}

<无正负号常量>::=<数字>{<数字>}

<字母>::=a|b|c|d|e|f|g|.....|u|v|w|x|y|z

<数字>::=0|1|2|3|4|5|6|7|8|9



1.2 编译程序的工作内容

- 程序设计语言的词法规则举例

$\langle \text{标识符} \rangle ::= \langle \text{字母} \rangle \{ \langle \text{字母} \rangle | \langle \text{数字} \rangle \}$

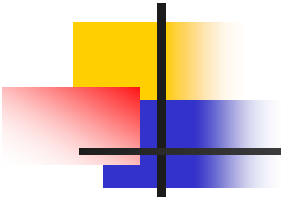
$\langle \text{无正负号常数} \rangle ::= \langle \text{数字} \rangle \{ \langle \text{数字} \rangle \}$

$\langle \text{字母} \rangle ::= a | b | c | \dots | x | y | z$

$\langle \text{数字} \rangle ::= 0 | 1 | 2 | \dots | 8 | 9$

词法规则有多种
表示形式

- EBNF (extended Backus-Naur form)



1.2 编译程序的工作内容

- 词法分析

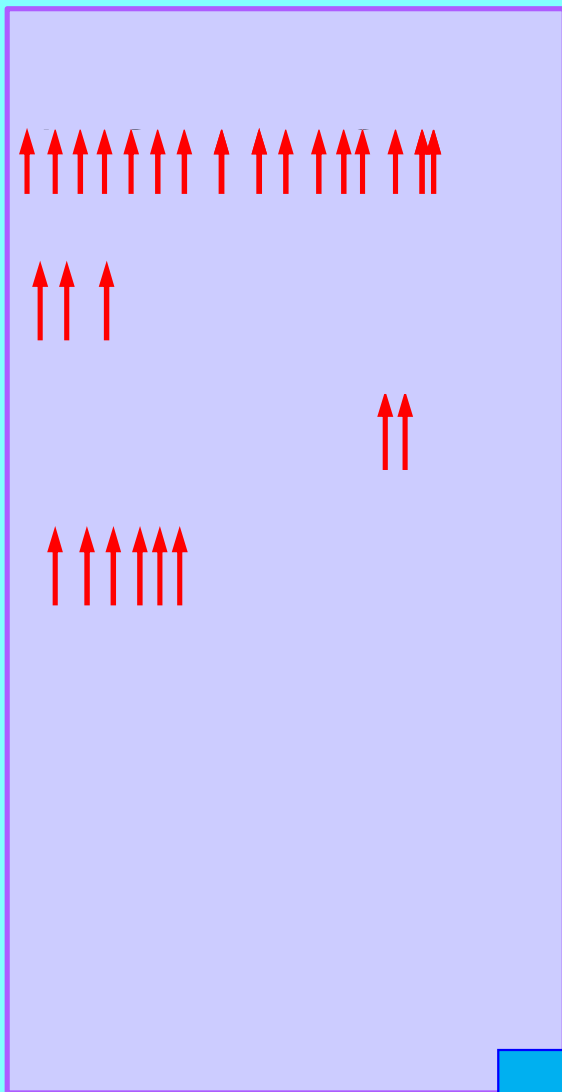
- 依据语言词法规则，分析由字符组成的源程序，把它识别为一个一个具有独立意义的**最小语法单位**，即“单词/词素(lexeme)”，并识别出与其**相关的属性**（如是标识符，是界限符，还是数，等等），再转换成**长度上统一的标准形式**（这种统一的标准形式既刻画了单词本身，又刻画了它所具有的属性，称为属性字token），以供其它部分使用。

< token-name, attribute-value >

```
program weight;  
begin  
    M:= 3.14159/4* (d1*d1-d2*d2)  
end.
```

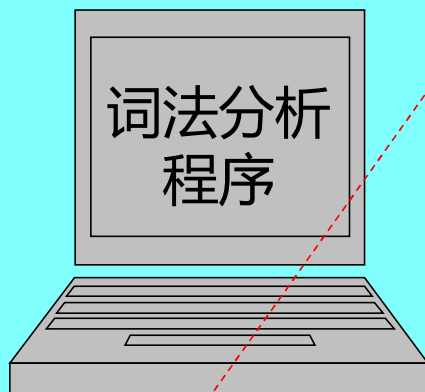
```
<标识符>::=<字母>{<字母>|<数字>}  
<无正负号常数>::=<数字>{<数字>}  
<字母>::=a|b|c|...|x|y|z  
<数字>::=0|1|2|...|8|9
```

源程序

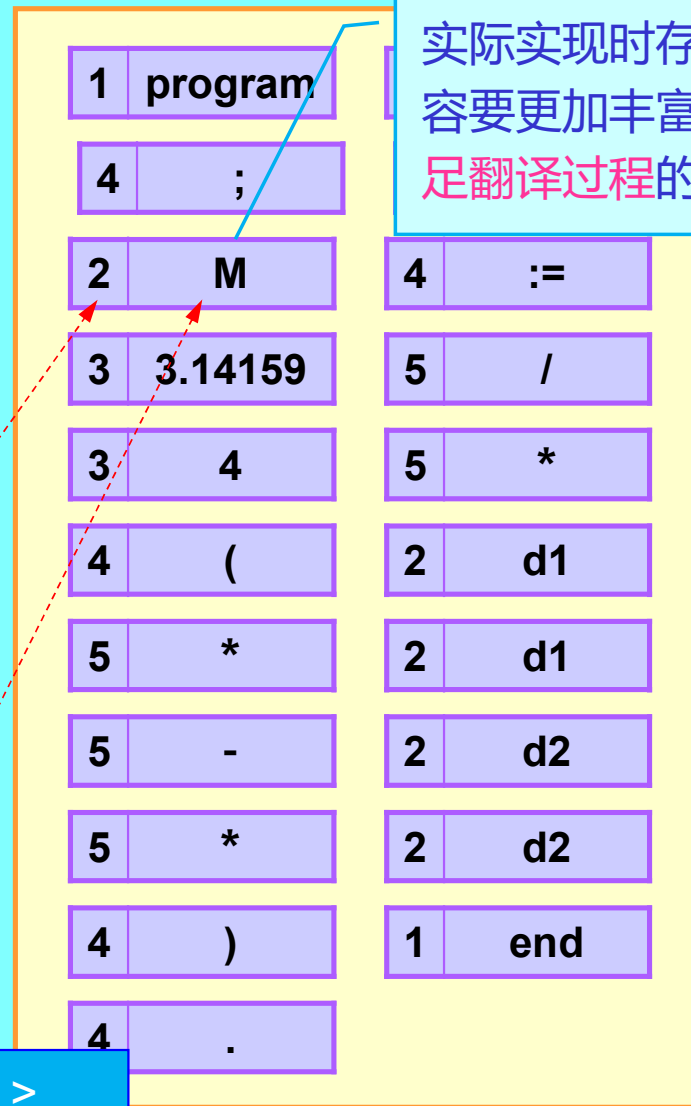


单词类别表

类别	编码
保留字	1
标识符	2
常数	3
界限符	4
运算符	5

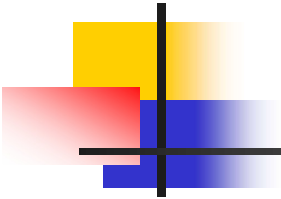


生成的单词串



实际实现时存放的内容要更加丰富——满足翻译过程的需要

< token-name, attribute-value >



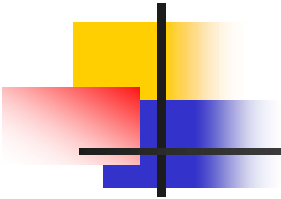
1.2 编译程序的工作内容

- 词法分析

- 依据语言词法规则，分析由字符组成的源程序，把它识别为一个一个具有独立意义的**最小语法单位**，即“单词/词素(lexeme)”，并识别出与其**相关的属性**（如是标识符，是界限符，还是数，等等），再转换成**长度上统一的标准形式**（这种统一的标准形式既刻画了单词本身，又刻画了它所具有的属性，称为属性字token），以供其它部分使用。

```
< token-name, attribute-value >
```

- 词法分析程序/扫描程序：scanner



1.2 编译程序的工作内容

- 语法分析

- 依据语法规则，逐一分析词法分析时得到的单词，把单词串分解成各类语法单位，即确定它们是怎样组成说明和语句，以及说明和语句又是怎样组成程序的。分析时如发现有不合语法规则的地方，便将出错的位置及出错性质打印报告给程序员；如无语法错误，则用另一种中间形式给出正确的语法结构，供下一阶段分析使用。
- 语法分析程序：parser

C语言中，一个程序由多个函数组成，一个函数由声明和语句组成，一个语句由表达式组成，……。

语法规则

G[程序]:

⟨程序⟩ ::= ⟨程序首部⟩ ; ⟨分程序⟩ .

⟨程序首部⟩ ::= program ⟨标识符⟩

⟨分程序⟩ ::= ⟨复合语句⟩

⟨复合语句⟩ ::= begin ⟨赋值语句⟩ end

⟨赋值语句⟩ ::= ⟨标识符⟩ := ⟨表达式⟩

⟨表达式⟩ ::= ⟨项⟩ { (+|-) ⟨项⟩ }

⟨项⟩ ::= ⟨因式⟩ { (*|/) ⟨因式⟩ }

⟨因式⟩ ::= ⟨标识符⟩ | ⟨无正负号常量⟩

⟨标识符⟩ ::= ⟨字母⟩ { ⟨字母⟩ | ⟨数字⟩ }

⟨无正负号常量⟩ ::= ⟨数字⟩ { ⟨数字⟩ }

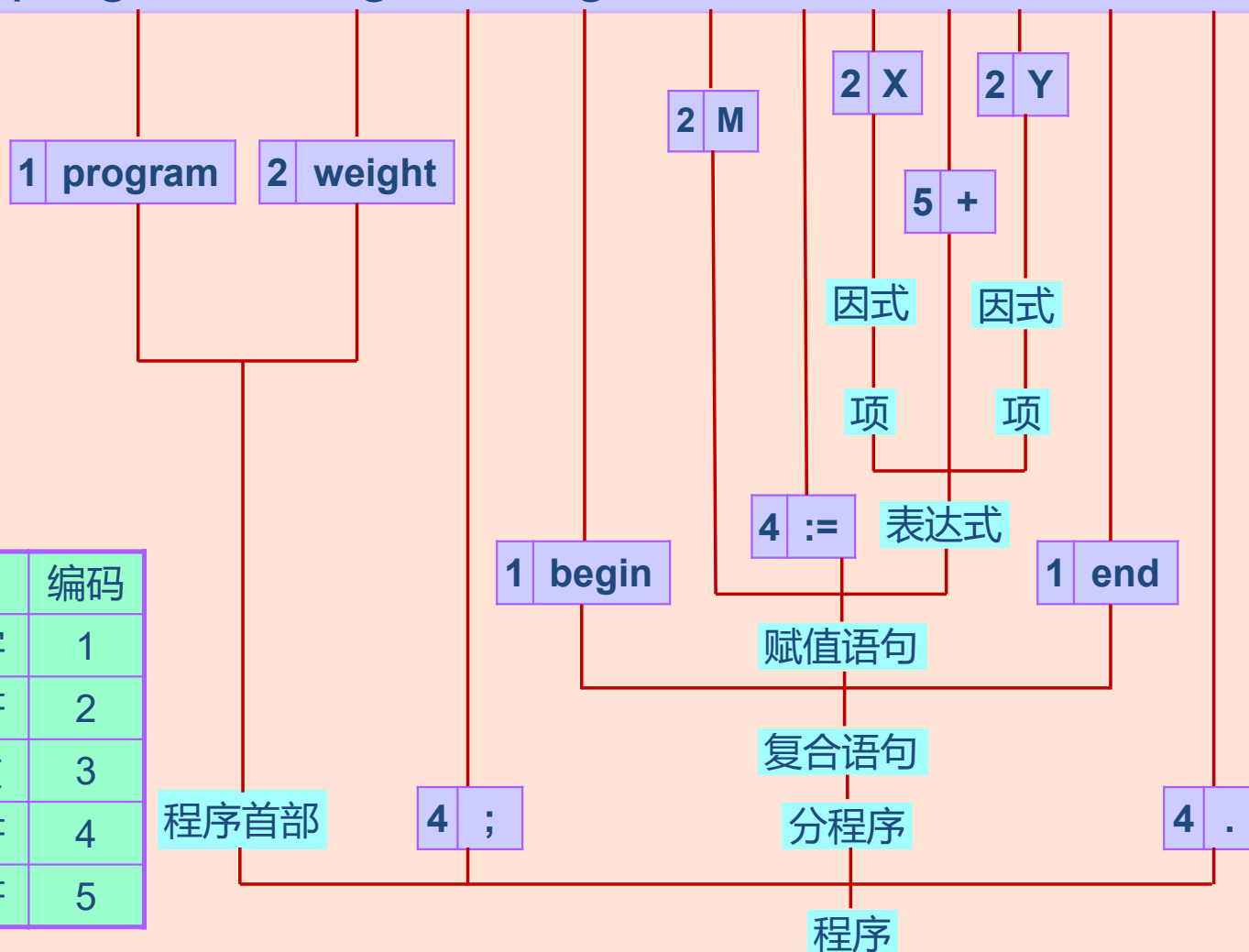
⟨字母⟩ ::= a|b|c|.....|x|y|z

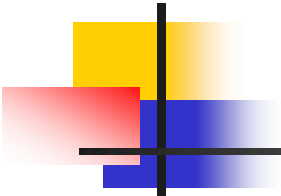
⟨数字⟩ ::= 0|1|2|3|4|5|6|7|8|9

类别	编码
保留字	1
标识符	2
常 数	3
界限符	4
运算符	5

实际是单词符号形式: < token-name, attribute-value >

program weight ; begin M := X + Y end .

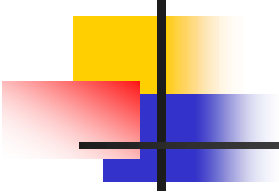




1.2 编译程序的工作内容

- 语法分析

- 依据语法规则，逐一分析词法分析时得到的单词，把单词串分解成各类语法单位，即确定它们是怎样组成说明和语句，以及说明和语句又是怎样组成程序的。分析时如发现有不合语法规则的地方，便将出错的位置及出错性质打印报告给程序员；如无语法错误，则用另一种中间形式给出正确的语法结构，供下一阶段分析使用。
- 语法分析程序：parser



1.2 编译程序的工作内容

- 语义分析

- 依据语言的语义规则，对语法分析得到的语法结构进行静态语义检查（确定类型、类型和运算合法性检查、识别含义与相应的语义处理及其它一些静态语义检查），并用另一种内部形式表示出来，或者直接用目标语言表示出来。
- 凡在编译时可以确定的内容称为“静态”的；凡必须推迟到程序运行时才能确定的内容称为“动态”的。

源程序: `program weight; begin M:= 3.14159/4*(d1*d1-d2*d2) end.`

生成的中间代码

1. 取3.14159
2. /4
3. 存W1
4. 取d1
5. *d1
6. 存W2
7. 取d2
8. *d2
9. 反减W2
10. *W1
11. 存M

运行阶段代码执行的模拟

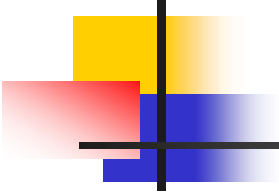
累加器内容: $(d1*d1 - d2*d2)*3.14159/4$

存储单元:

W1: 3.14159/4

W2: $d1*d1$

M: $(d1*d1 - d2*d2)*3.14159/4$

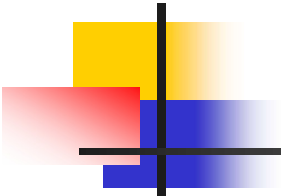


1.2 编译程序的工作内容

- 语义分析

- 依据语言的语义规则，对语法分析得到的语法结构进行静态语义检查（确定类型、类型和运算合法性检查、识别含义与相应的语义处理及其它一些静态语义检查），并用另一种内部形式表示出来，或者直接用目标语言表示出来。
- 凡在编译时可以确定的内容称为“静态”的；凡必须推迟到程序运行时才能确定的内容称为“动态”的。

- 语义分析程序：semantic analyzer



1.2 编译程序的工作内容

- 代码优化

- 依据程序的等价变换规则，尽量压缩目标程序运行所需的时间和所占的存储空间，以提高目标程序的质量。

更快、更短、能耗更低、可用性好.....

- 目标代码优化程序：target code optimizer

源程序: program weight; begin M:= 3.14159/4*(d1*d1-d2*d2) end.

优化生成的中间代码

1. 取d1
2. *d1
3. 存W1
4. 取d2
5. *d2
6. 反减W1
7. *0.78540
8. 存M

运行阶段代码执行的模拟

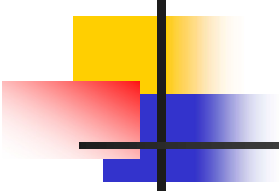
累加器内容: $(d1*d1 - d2*d2)*0.78540$

存储单元:

W1: $d1*d1$

W2:

M: $(d1*d1 - d2*d2)*0.78540$



1.2 编译程序的工作内容

- 代码生成

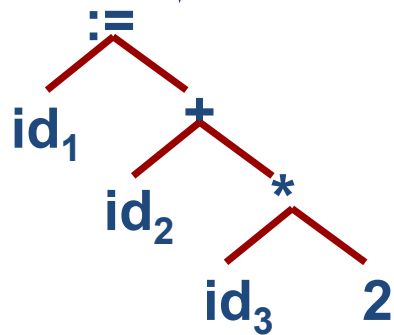
- 如果语义分析时把源程序表示成中间形式而不是表示成目标指令，则由本部分完成从中间形式到目标指令的转换。如果语义分析时，已直接生成目标指令，则无需另外再做代码生成工作。
- 目标指令可能是
 - 绝对指令代码，或者可重新定位的指令代码，或者汇编指令代码
- 该阶段的工作有赖于硬件系统结构和机器指令含义
- 代码生成程序：code generator

a:=b+c*2

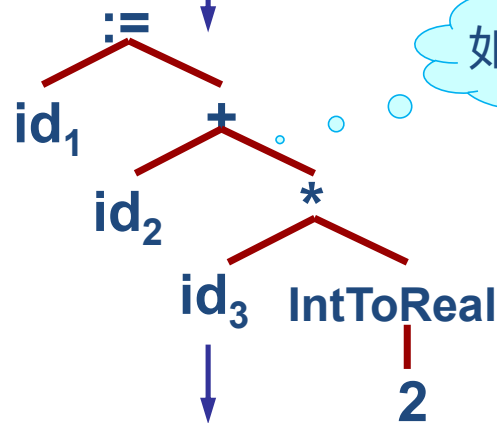
Lexical Analyses

id₁:=id₂+id₃*2

Syntax Analyses



Semantic Analyses



如何做检查?

Intermediate Generation

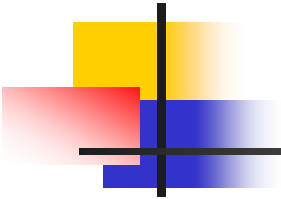
t₁:=IntToReal(2)
t₂:=id₃*t₁
t₃:=id₂+t₂
id₁:=t₃

Intermediate Optimization

t₁:=id₃* 2.0
id₁:=id₂+t₁

Code Generation

MOVF id₃, R₂
MULF #2.0, R₂
MOVF id₂, R₁
ADDF R₂, R₁
MOVF R₁, id₁



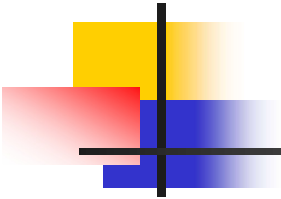
1.2 编译程序的工作内容

- 表格管理

- 登记源程序中出现的每个名字以及名字的各种属性
- 有些名字的属性需要在后续各个阶段才能填入

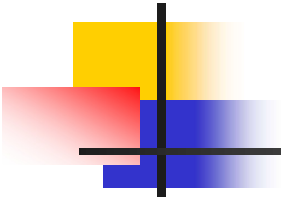
多种实现方式

- 符号表 (symbol table)



1.2 编译程序的工作内容

- 源程序中的错误
 - 语法错误
 - 不符合语法（或词法）规则的错误
 - 在词法分析或语法分析时检测出来
 - 语义错误
 - 静态语义错误一般在语义分析时检测出来
 - 动态语义错误要在运行时才能检测出来
 - ✓ 如逻辑错误，通常包括
 - 说明错误、作用域错误、类型不一致等



1.2 编译程序的工作内容——小结

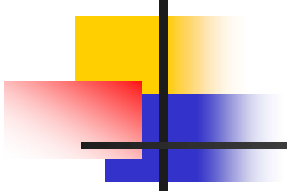
- 编译程序的目的

- 将高级语言的源程序翻译成等价的低级语言程序



- 编译程序的工作内容

- 功能：词法分析、语法分析、语义分析、中间代码优化、目标代码生成
- 实现：表格管理、出错处理

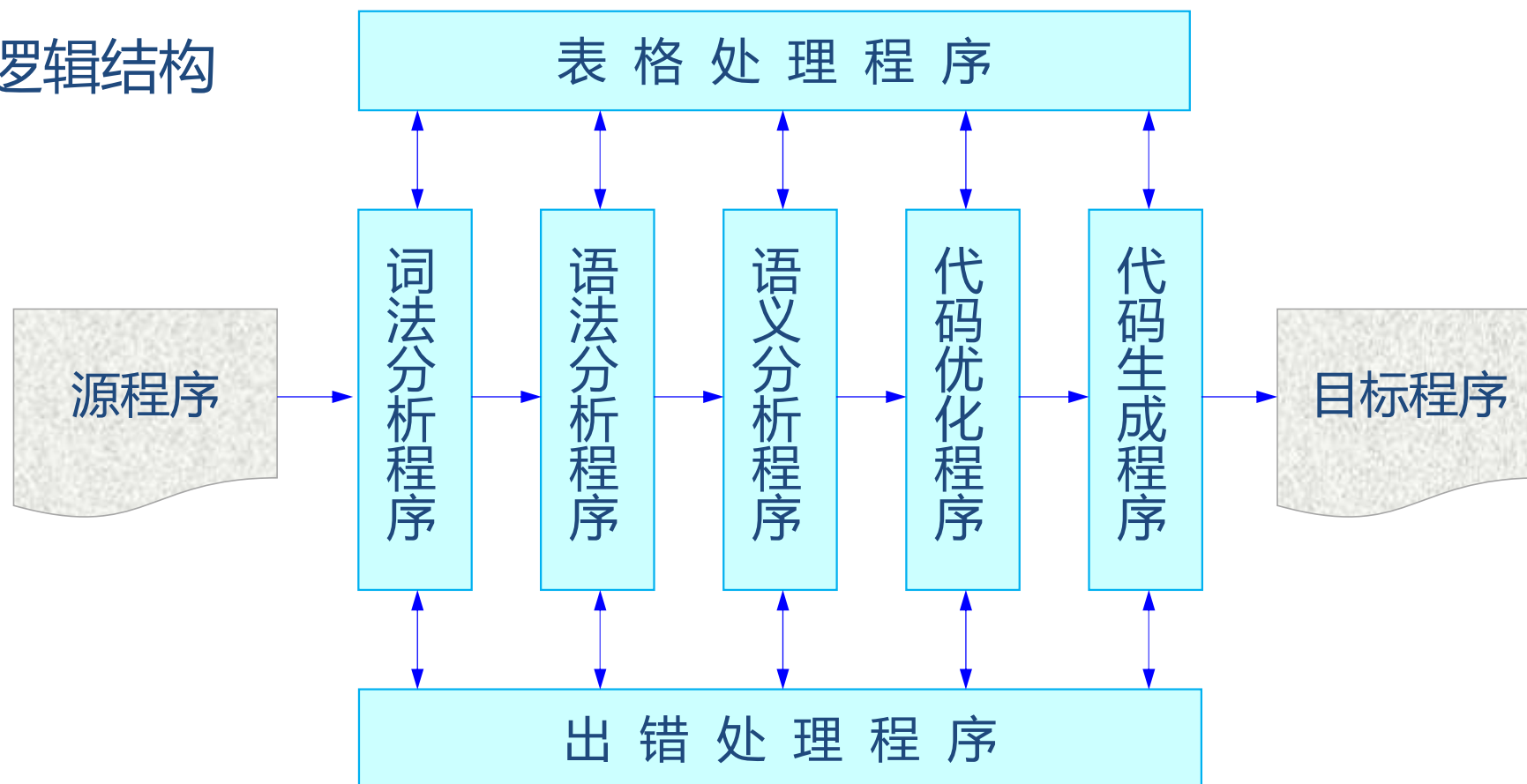


本章内容

- ◆ 程序的翻译
- ◆ 编译程序的工作内容
- ◆ 编译程序的结构
- ◆ 编译程序的组织方式
- ◆ 编译程序的构造

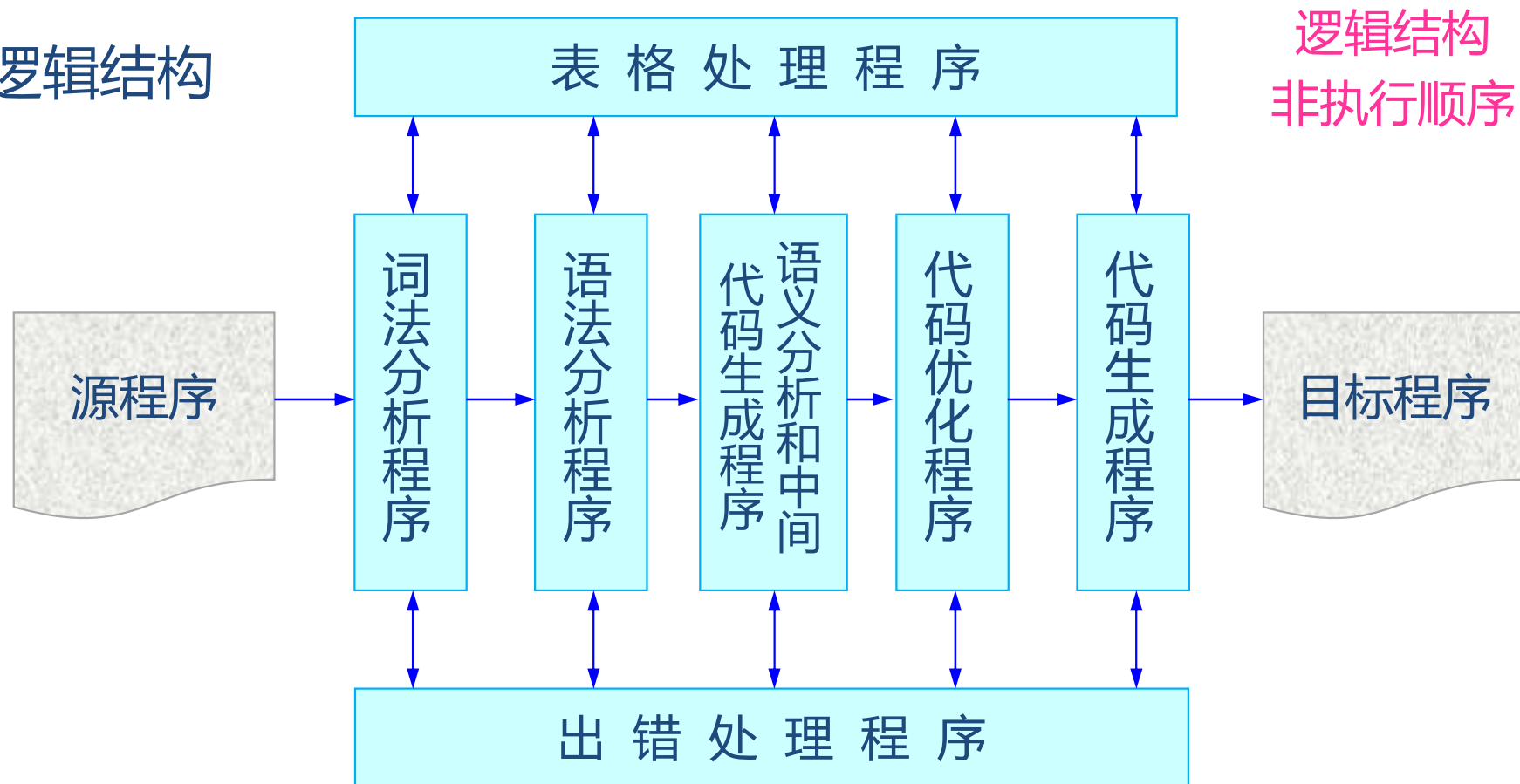
1.3 编译程序的结构

- 逻辑结构



1.3 编译程序的结构

- 逻辑结构



编译程序的结构

逻辑结构
非执行顺序

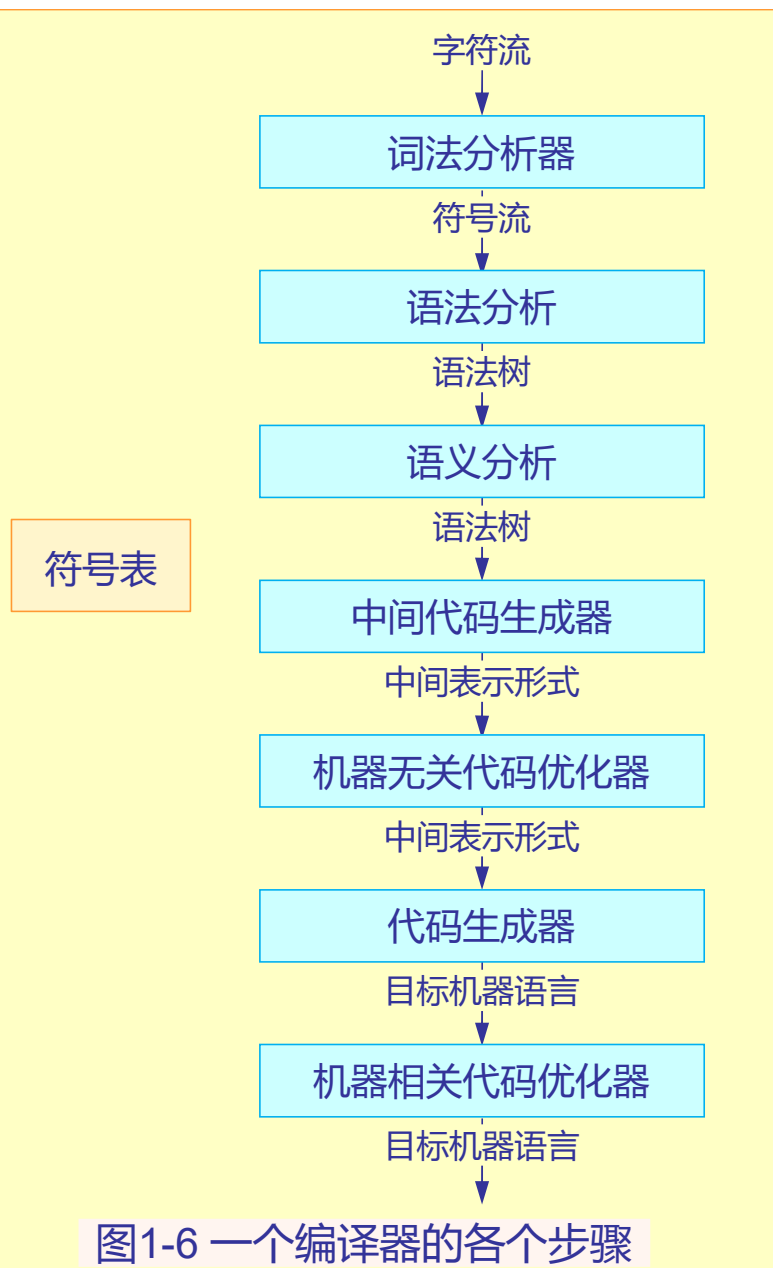
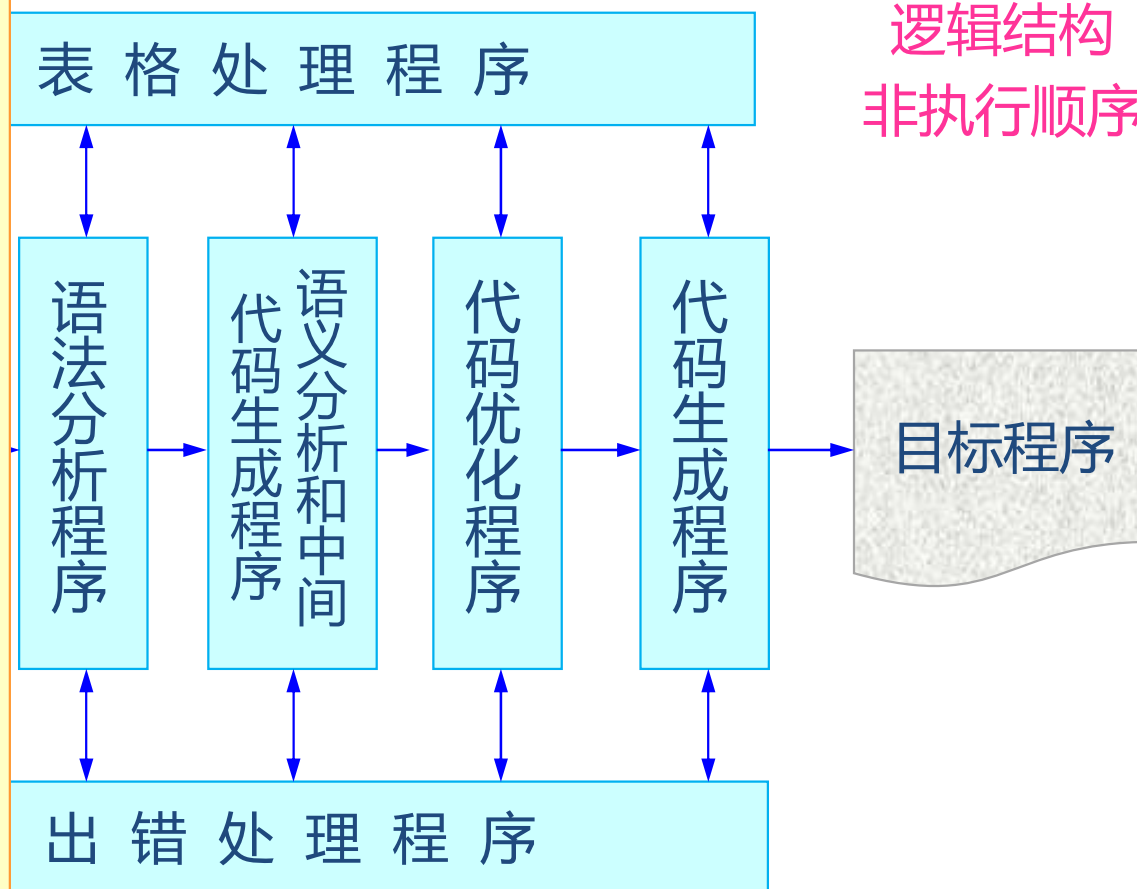
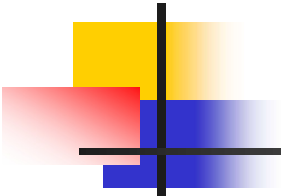


图1-6 一个编译器的各个步骤



1.3 编译程序的结构

- 编译程序的“分析—综合”模型

- 分析部分

- 词法分析 (Lexical Analyses)
 - 语法分析 (Syntax Analyses)
 - 语义分析 (Semantic Analyses)

准确、通顺

把一事物、一种现象、一个概念分成各个部分，找出这些部分的本质属性和彼此之间的关系。

- 综合阶段

- 中间代码生成 (Intermediate Generation)
 - 代码优化 (Intermediate Optimization)
 - 代码生成 (Code Generation)

转换、陈述

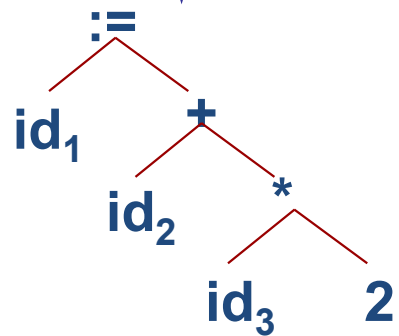
将不同部分、不同事物的属性合并成为一个整体来对待。

a:=b+c*2

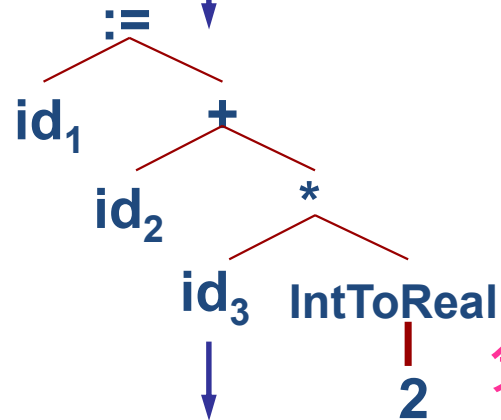
Lexical Analyses

id₁:=id₂+id₃*2

Syntax Analyses



Semantic Analyses



Intermediate Generation

t₁:=IntToReal(2)
t₂:=id₃*t₁
t₃:=id₂+t₂
id₁:=t₃

Intermediate Optimization

t₁:=id₃* 2.0
id₁:=id₂+t₁

Code Generation

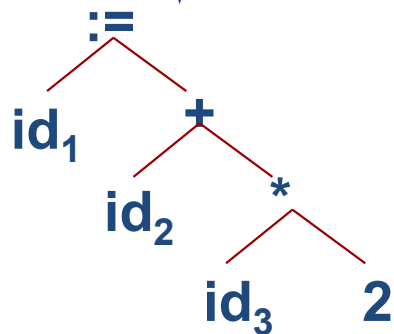
MOVF id₃, R₂
MULF #2.0, R₂
MOVF id₂, R₁
ADDF R₂, R₁
MOVF R₁, id₁

a:=b+c*2

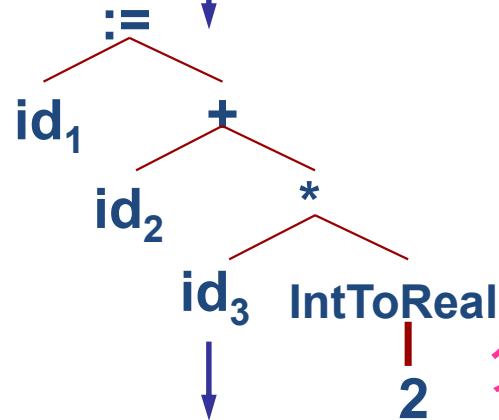
Lexical Analyses

id₁:=id₂+id₃*2

Syntax Analyses



Semantic Analyses



Intermediate Generation

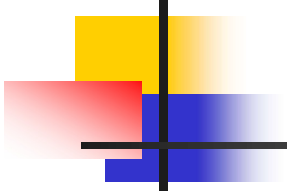
t₁:=IntToReal(2)
t₂:=id₃*t₁
t₃:=id₂+t₂
id₁:=t₃

Intermediate Optimization

把一事物、一种现象、一个概念分成各个部分，找出这些部分的本质属性和彼此之间的关系。

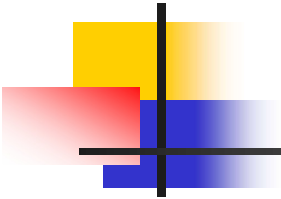
将不同部分、不同事物的属性合并成为一个整体来对待。

MOVF id₂, R₁
ADDF R₂, R₁
MOVF R₁, id₁



本章内容

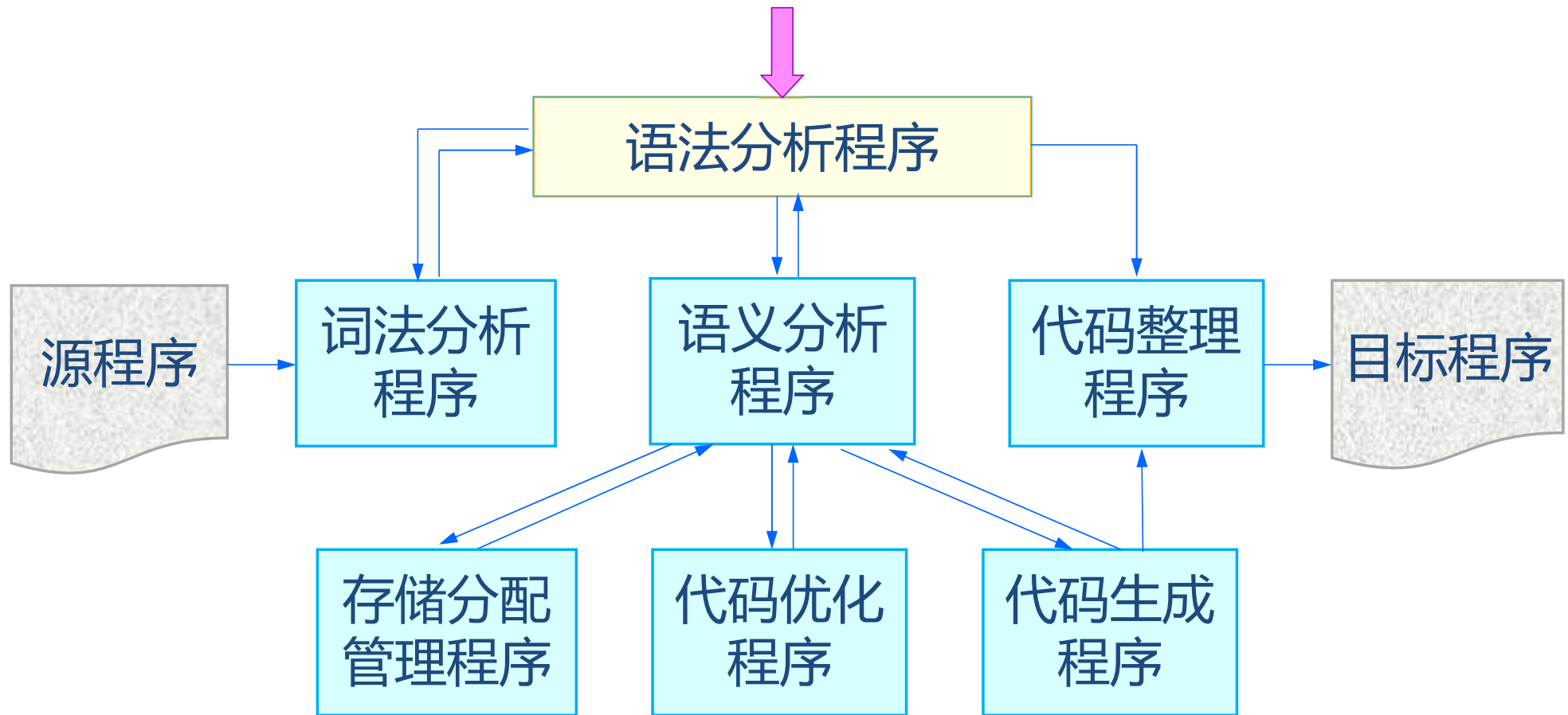
- ◆ 程序的翻译
- ◆ 编译程序的工作内容
- ◆ 编译程序的结构
- ◆ 编译程序的组织方式
- ◆ 编译程序的构造



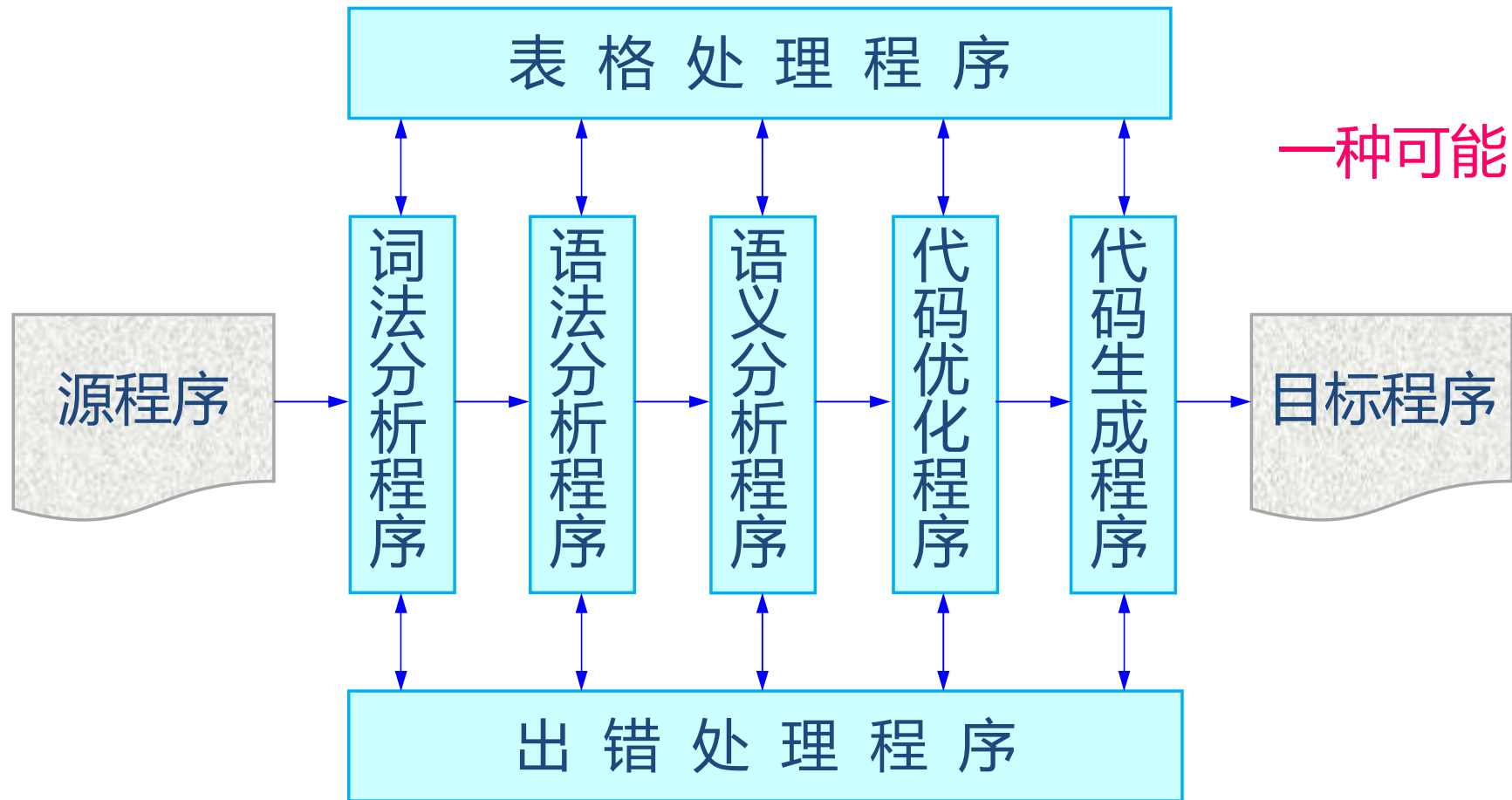
1.4 编译程序的组织形式

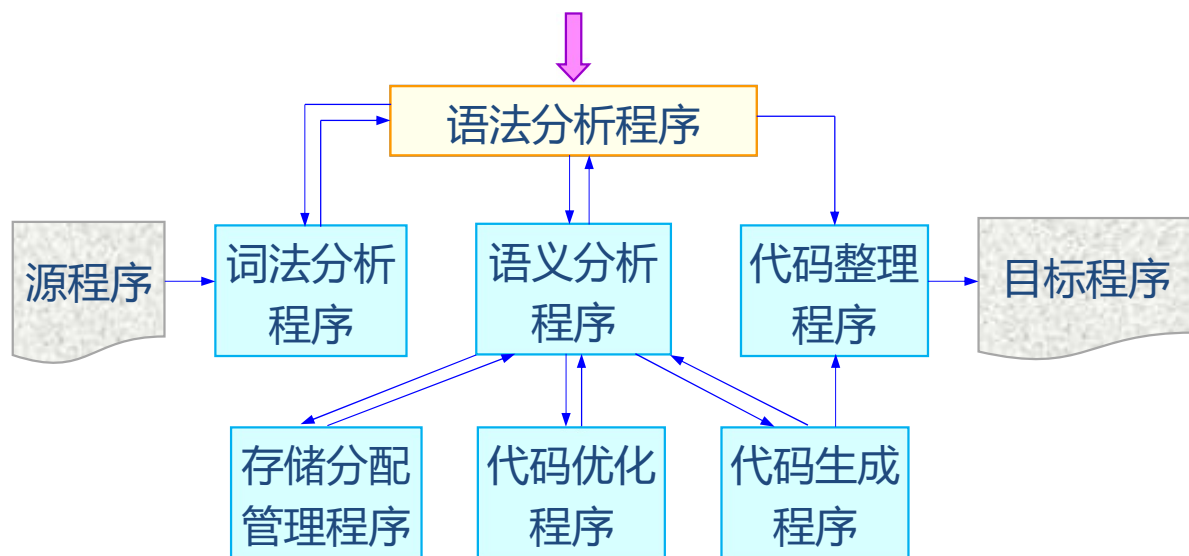
- 遍(趟, 趟程, pass)
 - 一个编译程序在编译时刻把源程序或源程序的等价物(中间程序)从头到尾扫描一遍并转换成另一紧邻的等价物的全过程
 - 根据编译程序在完成翻译任务的过程中需要对源程序或其中间等价物扫描的遍数, 可以把编译程序分为单遍扫描的编译程序(只需扫描一遍)和多遍扫描的编译程序(需扫描多遍)

单遍扫描的编译程序

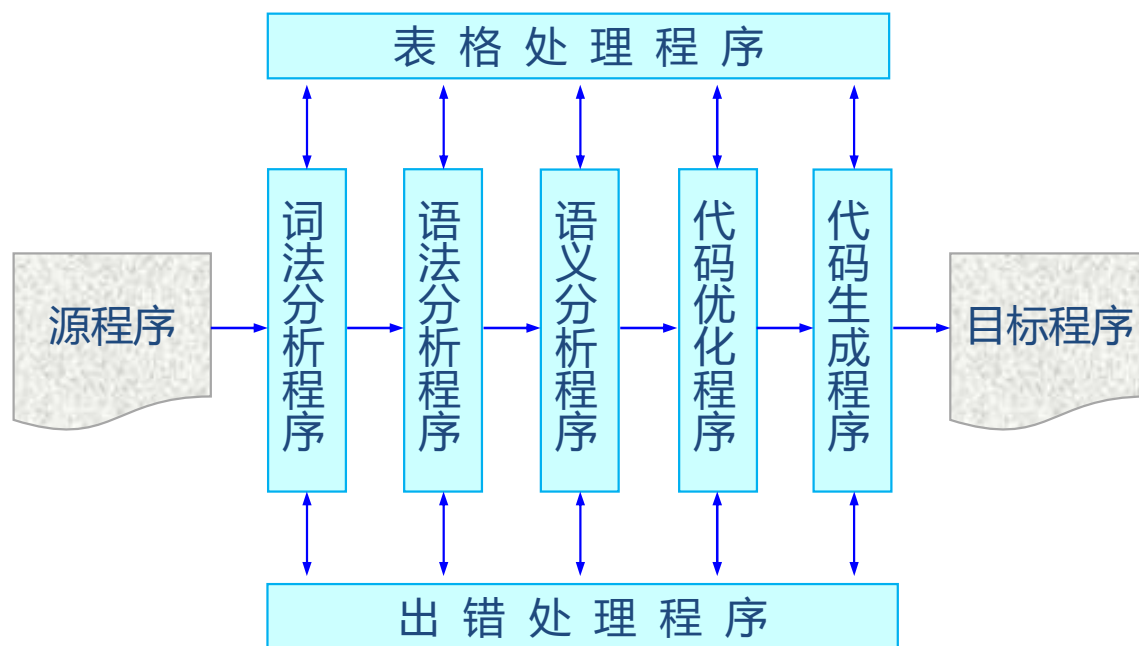


多遍扫描的编译程序





采用单遍扫描
还是多遍扫描?



1.4 编译程序的组织形式

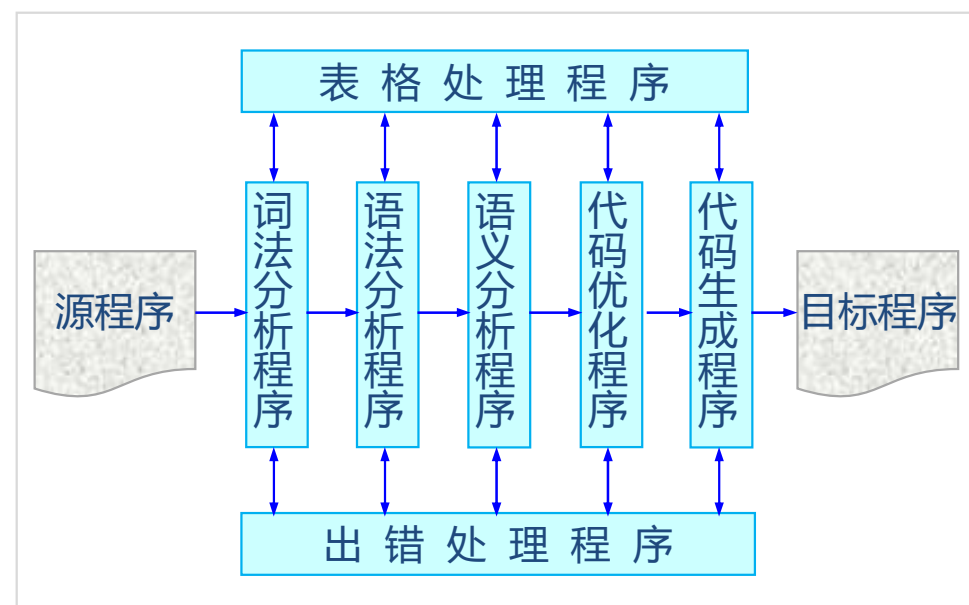
- 分遍的优点

- 结构清晰、节省内存

- 每遍都从外存获取前一遍的结果作为开始，工作结果仍记入外存，每遍几乎可使用全部内存

- 考虑因素

- 计算机内存容量
 - 源语言的繁简与特点
 - 编译程序开发人员的情况
 -



1.4 编译程序的组织形式

- 从移植的角度考虑

- 编译的前端(front end)

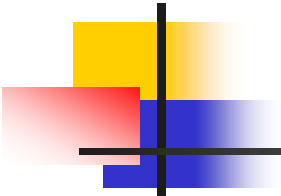
- 由与源语言有关但与目标机器无关的那些部分组成，如词法分析、语法分析、语义分析与中间代码生成及部分代码优化工作

- 编译的后端(back end)

- 包括编译中与目标机器有关的那些部分，如与目标机有关的代码优化和目标代码生成等。后端不依赖于源语言而仅依赖于中间语言

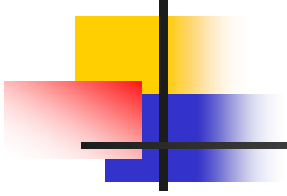
可通过改变编译程序的后端实现编译程序的移植





目标代码运行平台

- 目标平台
 - 编译器生成的代码所运行的系统平台
 - 输出的目标代码在虚拟机上运行
 - 编译器和运行平台有可能相同，也有可能不同
- 本地编译器
 - 输出的目标代码与编译器运行在相同类型的计算机和操作系统之上
 - 开发嵌入式系统通常需要使用交叉编译器
- 交叉编译器
 - 输出的目标代码可以与编译器运行在不同的平台之上



本章内容

- ◆ 程序的翻译
- ◆ 编译程序的工作内容
- ◆ 编译程序的结构
- ◆ 编译程序的组织方式
- ◆ 编译程序的构造

1.5 编译程序的构造

- 构造编译程序可以用机器语言、汇编语言和高级语言

如何构造一个编译程序？*

- 一个编译程序也可以有“前世今生”

- 编译程序的高级语言源程序形式

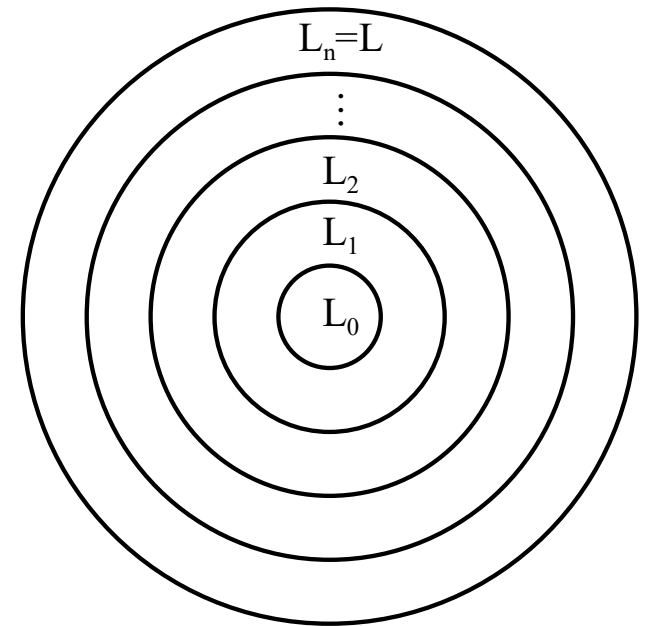


- 编译程序的机器代码形式



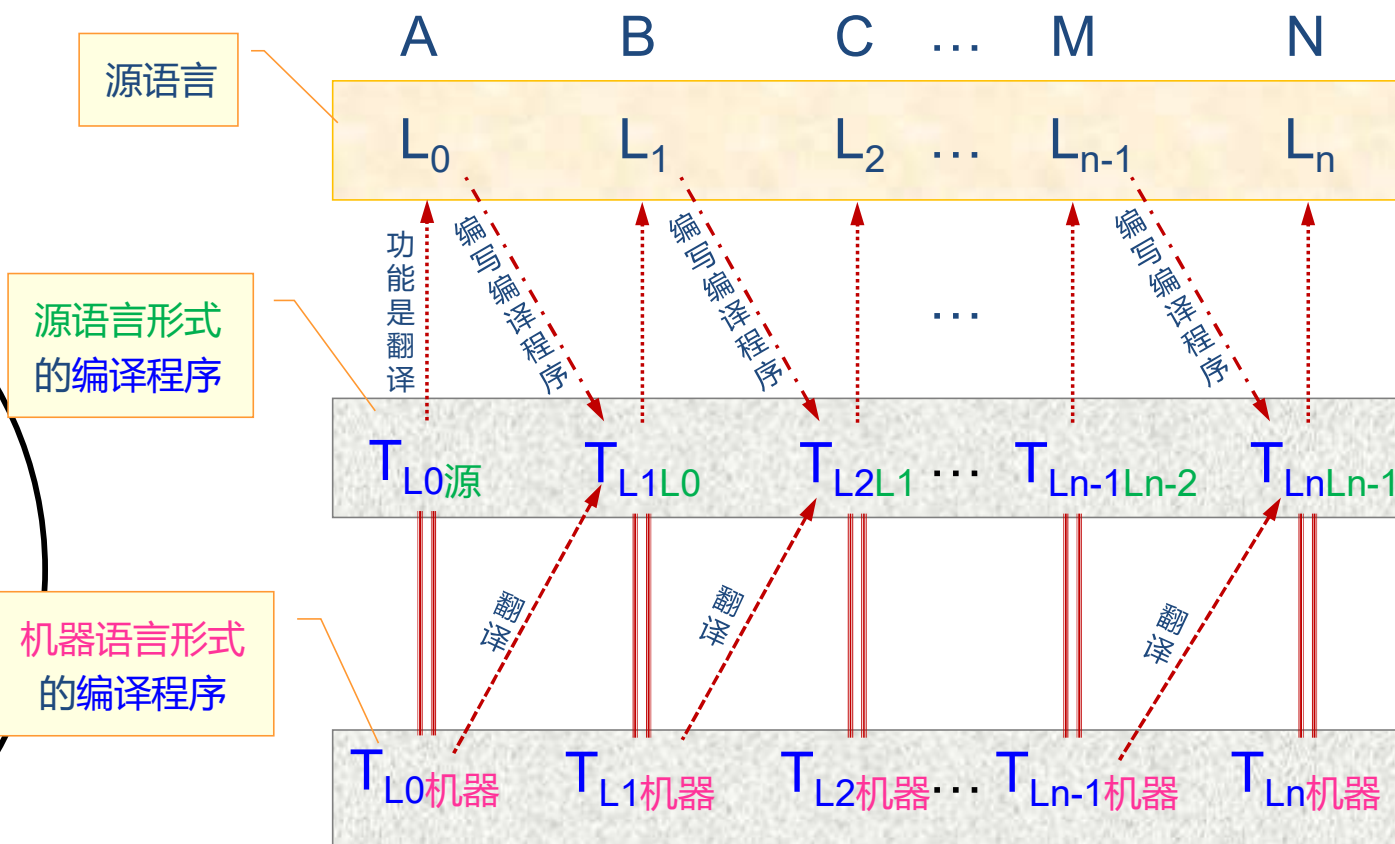
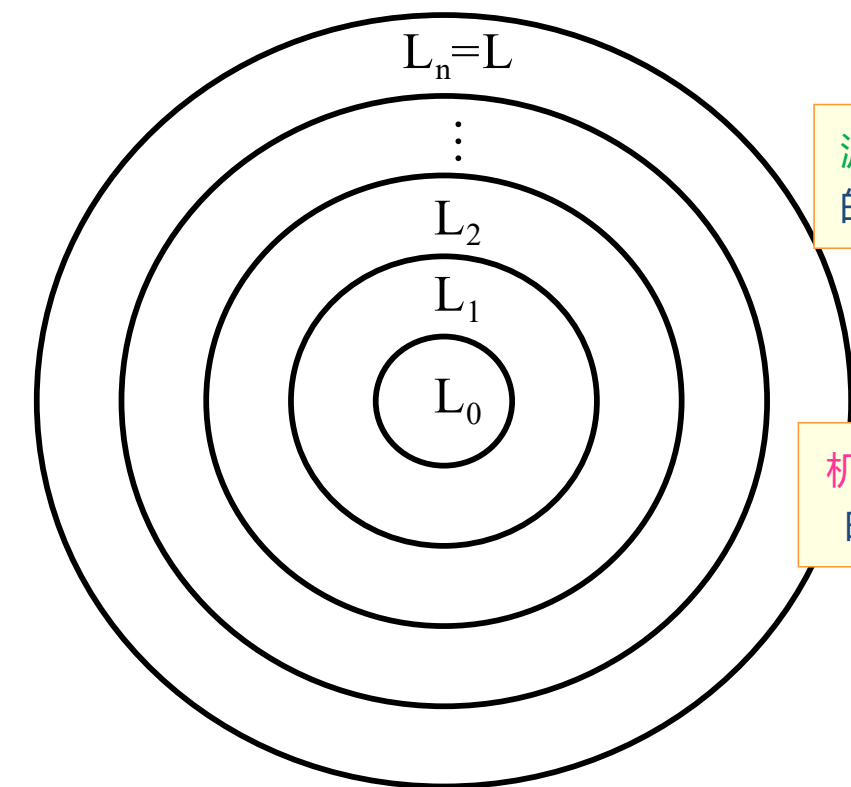
1.5 编译程序的构造

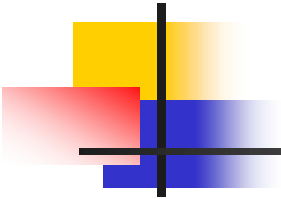
- 高级语言的自编译性
 - 一个语言可以用来编写自己的编译程序
- 编译的自展技术
 - 通过一系列自展途径而形成编译程序的过程
 - 先构造语言核心部分的编译程序(可低级语言实现)
 - 再以它为工具构造能够编译更多语言成分的较大编译程序
 - 如此扩展下去, 越滚越大, 最后形成所期望的整个语言的编译程序





L 语言的核心部分与扩充部分的翻译程序构造





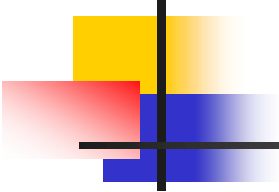
1.5 编译程序的构造

- 编译的移植

- 将一个机器(宿主机)上的一个具有自编译性的高级语言编译程序移植到另一个机器(目标机)上

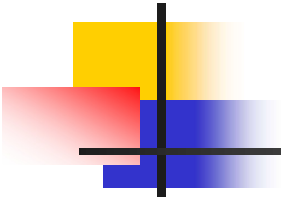
- 将 A 计算机系统中高级语言 L 的编译程序移植到 B 计算机系统中





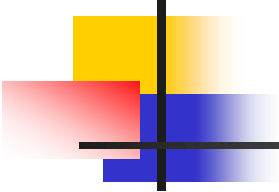
1.5 编译程序的构造

- 编译程序自动化
 - 编译器构造工具
 - 语法分析器的生成器
 - 扫描器的生成器
 - 语法制导的翻译引擎
 - 代码生成器的生成器
 - 数据流分析引擎
 - 编译器构造工具集
 -



1.5 编译程序的构造

- UNIX操作系统中的两个工具
 - Lex——词法分析程序生成器
 - YACC——语法分析程序生成器
- Windows操作系统中的相应工具
 - Flex——词法分析程序生成器
 - Bison——语法分析程序生成器



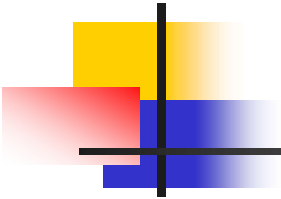
1.6 与编译器相关的程序

- 解释程序(interpreter)
- 汇编程序(assembler)
- 连接程序(linker)
- 装入程序(loader)
- 预处理程序(preprocessor)
- 编辑程序(editor)
- 调式程序(debugger)
- 描述器(profiler)
- 项目管理程序(project manager)

【 Louden 】 《编译原理及实践》
“1.2 与编译器相关的程序”

【ALSU】 龙书
“1.1 语言处理程序”

语言处理器：一个集成的软件开发环境，其中包括很多种类的语言处理器，比如编译器、解释器、汇编器、连接器、加载器、调试器以及程序概要提取工具。



1.7 编译课程的学习定位

高级语言
源程序

编译程序

汇编语言或机器
语言的目标程序

- 实现编译程序的必备知识
 - 源语言知识 —— 高级语言程序设计
 - 目标语言知识 —— 计算机组成、汇编语言
 - 翻译方法 —— 编译原理

1.7 编译课程的学习定位

高级语言
源程序

编译程序

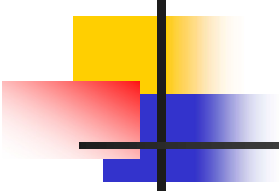
汇编语言或机器
语言的目标程序

- 目的：程序语言特点与目标体系结构特征匹配*
- 本质：算法的设计
 - 与其它算法课程的不同
 - 数据结构、算法设计——通用算法设计
 - 编译原理——“专用”算法设计

AI 时代
的挑战？

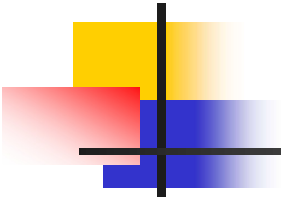
编译器的设计者：

- 跟踪新的语言特征
- 设计出新的翻译算法
 - 尽可能利用新硬件的能力



本章内容小结

- 程序的翻译：源语言、目标语言、等价；编译方式、解释方式
- 编译方式的特点：分阶段、生成目标代码
- 解释方式的特点：动态顺序
- 编译程序的工作：词法分析、语法分析、语义分析、代码优化、目标代码生成
- 编译程序的结构：逻辑结构；分析-综合模型
- 编译程序的组织形式：遍；前后端
- 编译程序的构造方法：直接编写、自编译、移植、自动工具



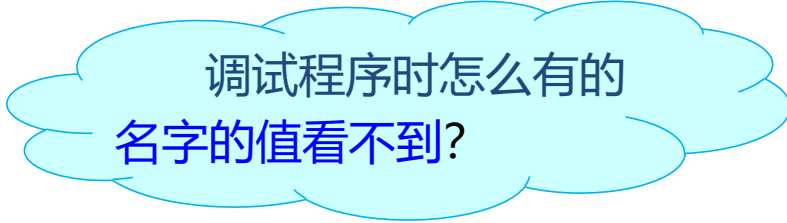
学习目标

- 重点掌握

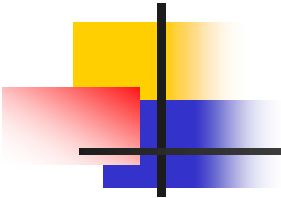
- 能叙述程序翻译的基本思想，并能用其分析、解释相关问题
- 能叙述两种程序翻译方式及其特点，并能用其分析、解释相关问题
- 能叙述编译程序及其工作内容，并能用其分析、解释相关问题
- 能叙述编译程序的结构，并能用其分析、解释相关问题

- 一般了解

- 能叙述编译程序的构造方法



调试程序时怎么有的
名字的值看不到？



参考文献

- 《编译原理：原理、技术和工具》(第2版), Alfred V.Aho等, 机械工业出版社, 2008
- 《编译原理与实践》, Kenneth C.Louden, 机械工业出版社, 2004
- 《程序设计语言编译原理》(第3版), 陈火旺等, 国防工业出版社, 2000
- 《编译原理》(第3版), 何炎祥等, 华中科技大学出版社, 2010
- 《编译方法》(修订版), 胡笔蕊等, 电子工业出版社, 1994
-